

Assignment Kit for Coding Standard



Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

This material is approved for public release.
Distribution limited by the Software Engineering Institute to attendees.

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview This assignment kit covers the following topics.

Section	See Page
Prerequisites	2
Objectives	2
Coding standard requirements	3
Example coding standard	4
Evaluation criteria and suggestions	7
Coding standard template	8

Prerequisites

Prerequisites

- Read Chapter 4
- Complete Size Counting Standard

Objectives

The objectives of the coding standard are to

- establish a consistent set of coding practices
- provide criteria for judging the quality of the code that you produce
- facilitate size counting by ensuring your programs are written so they can be readily counted
- for LOC counting, require that there be a separate physical line for each logical line of code

Coding standard requirements

Coding standard requirements

Produce, document, and submit a completed coding standard that calls for quality coding practices.

For LOC counting, ensure that a separate physical source line is used for each logical line of code.

Submit the coding standard with your program 2 assignment package.

Example coding Standard

Coding standard example

Pages 5 and 6 of this workbook contain an example C++ coding standard.

Notes about the example

- Since it is an example, tailor it to meet your personal needs.
 - If you have an existing organizational standard, consider using it for the PSP exercises.
-

Continued on next page

Example C++ Coding Standard

Purpose	To guide implementation of C++ programs
Program Headers	Begin all programs with a descriptive header.
Header Format	<pre> /***** /* Program Assignment: the program number */ /* Name: your name */ /* Date: the date you started developing the program */ /* Description: a short description of the program and what it does */ *****/ </pre>
Listing Contents	Provide a summary of the listing contents
Contents Example	<pre> /***** /* Listing Contents: /* Reuse instructions /* Modification instructions /* Compilation instructions /* Includes /* Class declarations: /* CData /* ASet /* Source code in c:/classes/CData.cpp: /* CData /* CData() /* Empty() *****/ </pre>

(continued)

Example C++ Coding Standard (continued)

Reuse Instructions	- Describe how the program is used: declaration format, parameter values, types, and formats. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Instruction Example	<pre> /***** /* Reuse instructions */ /* int PrintLine(char *line_of_character) */ /* Purpose: to print string, 'line_of_character', on one print line */ /* Limitations: the line length must not exceed LINE_LENGTH */ /* Return 0 if printer not ready to print, else 1 */ *****/ </pre>
Identifiers	Use descriptive names for all variable, function names, constants, and other identifiers. Avoid abbreviations or single-letter variables.
Identifier Example	<pre> Int number_of_students; /* This is GOOD */ Float: x4, j, ftave; /* This is BAD */ </pre>
Comments	- Document the code so the reader can understand its operation. - Comments should explain both the purpose and behavior of the code. - Comment variable declarations to indicate their purpose.
Good Comment	<pre>If(record_count > limit) /* have all records been processed? */</pre>
Bad Comment	<pre>If(record_count > limit) /* check if record count exceeds limit */</pre>
Major Sections	Precede major program sections by a block comment that describes the processing done in the next section.
Example	<pre> /***** /* The program section examines the contents of the array 'grades' and calcu- */ /* lates the average class grade. */ *****/ </pre>
Blank Spaces	- Write programs with sufficient spacing so they do not appear crowded. - Separate every program construct with at least one space.
Indenting	- Indent each brace level from the preceding level. - Open and close braces should be on lines by themselves and aligned.
Indenting Example	<pre> while (miss_distance > threshold) { success_code = move_robot (target_location); if (success_code == MOVE_FAILED) { printf("The robot move has failed.\n"); } } </pre>
Capitalization	- Capitalize all defines. - Lowercase all other identifiers and reserved words. - To make them readable, user messages may use mixed case.
Capitalization Examples	<pre> #define DEFAULT-NUMBER-OF-STUDENTS 15 int class-size = DEFAULT-NUMBER-OF-STUDENTS; </pre>

Evaluation criteria and suggestions

Evaluation criteria

Your standard must be

- complete
 - legible
-

Suggestions

Keep your standards simple and short.

Do not hesitate to copy or build on the PSP materials.

Coding Standard Template

Purpose	Guiar el desarrollo de programas Java manteniendo consistencia, legibilidad y facilitando el conteo de líneas de código.
Program Headers	Comenzar todos los programas con un header descriptivo.
Header Format	<pre>/** * Asignación de Programa: PSP 2A * Nombre: [Karla Sofía Castro Pérez] * Fecha: [fecha-actual] * Descripción: [Breve descripción de la clase y su funcionalidad] */</pre>
Listing Contents	Proveer un resumen del contenido del listado indicando clases, métodos principales y estructura.
Contents Example	<pre>/** * Program Assignment: PSP 2A * Name: [Karla Sofía Castro Pérez] * Date: [2025-06-11] * Description: Clase lógica principal que coordina el conteo de líneas y métodos */</pre>
Reuse Instructions	• Describir cómo usar la clase: parámetros, valores de retorno, excepciones • Especificar dependencias y requisitos del sistema • Documentar limitaciones y condiciones de error
Reuse Example	<pre>/** * Lee el contenido de un archivo y lo retorna como String * @param inFile nombre del archivo a leer * @return contenido del archivo como String, o string vacío en caso de error */ public String readData(String inFile) { }</pre>
Identifiers	Usar nombres descriptivos para todas las variables, nombres de función, constantes y otros identificadores. Evitar abreviaciones o variables de una sola letra.
Identifier Example	<pre>String fileName; // BUENO - descriptivo Scanner scanner; // BUENO - claro propósito String fn; // MALO - abreviado String x; // MALO - no descriptivo</pre>

(continued)

Coding Standard Template (continued)

Comments	• Documentar el código para que el lector pueda entender su operación • Los comentarios deben explicar tanto el propósito como el comportamiento del código • Comentar declaraciones de variables para indicar su propósito
Good Comment	<pre>// Crear scanner para leer entrada del usuario (Logic2a.java) Scanner scanner = new Scanner(System.in); // Leer el contenido del archivo especificado por el usuario (Logic2a.java) String fileContent = myInput.readData(fileName);</pre>
Bad Comment	<pre>// crear sc Scanner sc = new Scanner(System.in); // leer archivo String fc = input.readData(fn);</pre>
Major Sections	Preceder las secciones principales del programa por un comentario de bloque que describa el procesamiento realizado en la siguiente sección.
Example	<pre>// Crear instancias de las clases necesarias para el análisis (Logic2a.java) Input myInput = new Input(); // Para leer archivos Data myData = new Data(); // Para procesar datos LineCounter myLineCounter = new LineCounter(); // Para contar líneas</pre>
Blank Spaces	• Escribir programas con suficiente espaciado para que no aparezcan amontonados • Separar cada constructo del programa con al menos un espacio
Indenting	• Sangrar cada nivel de llave desde el nivel precedente • Las llaves de apertura y cierre deben estar en líneas separadas y alineadas
Indenting Example	<pre>public int count(String[] artData) { int count = 0; for (String line : artData) { String trimmedLine = line.trim(); if (trimmedLine.isEmpty() trimmedLine.startsWith("/") trimmedLine.equals("{") trimmedLine.equals("}")) { continue; } count++; } return count; }</pre>

Capitalization	• Capitalizar nombres de clases: LineCounter, MethodCounter • Usar camelCase para métodos y variables: countLines(), fileName • Usar MAYÚSCULAS para constantes: MAX_LINES, DEFAULT_FILE
Capitalization Example	<pre> public class LineCounter { // Class - PascalCase private int count; // Variable - camelCase public int count(String[] artData) { // Method - camelCase // Implementation from your actual code } } </pre>